

01 · 논리 게이트 실습

1. 새 창에서 실습 workspace를 연다.
2. AND·OR·XOR 소스와 테스트벤치를 읽는다.
3. 네 입력 조합을 시뮬레이션하고 파형을 해석한다.
4. 실험 전 레포트를 작성한 뒤 Vivado GUI를 연다.

앞부분: 새로 작성한 VS Code 사전 실습

뒷부분: 레거시 교안의 Vivado 절차와 원본 화면

작성일 2026. 09. 10. 이해리

01 · 실습 목차

1. 준비: 도구·새 창·workspace
2. 확장 확인과 코드 열기
3. 예상값·RTL·테스트벤치 읽기
4. 시뮬레이션 작업 실행과 PASS 확인
5. VaporView: 파형 열기·신호 추가·값 읽기
6. 실험 전 레포트와 문제 해결
7. 그다음: Vivado 프로젝트·시뮬레이션·FPGA

좌하단 **목차** 버튼은 언제든지 이 페이지로 돌아온다.

시작 전에 준비할 것

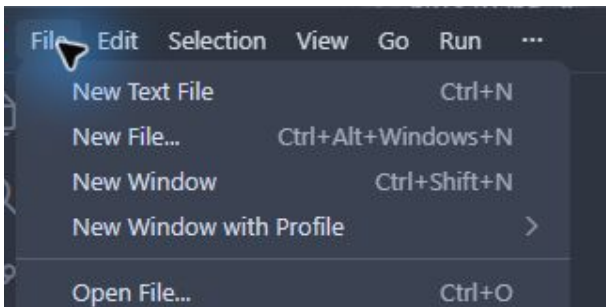
- Windows PC에 VS Code와 Vivado 시뮬레이션 도구가 설치되어 있어야 한다. 저장소도 PC에 내려받는다.
- VS Code는 편집기다. 이번 사전 실습의 실제 시뮬레이터는 XSim, 파형 뷰어는 VaporView다.
- xvlog, xelab, xsim의 실행 경로를 준비한다. Vivado GUI나 .xpr를 먼저 열 필요는 없다.
- 설치가 안 되어 있으면 같은 배포 폴더의 02.vscodex_verilog_환경설정_매뉴얼.pdf를 먼저 따른다.

이번 캡처 검증 환경: XSim 2026.1, slang 확장 0.2.17, VaporView 1.5.4, 사용자의 Nord 테마.

뒤의 레거시 화면과 도구 버전이 다르며, 레거시 Vivado·실제 보드의 새 실행 증빙을 뜻하지 않는다.

1. VS Code 새 창부터 열기

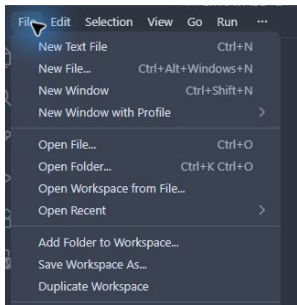
File → New Window를 누른다. 단축키는 Ctrl+Shift+N이다.



기존 창은 그대로 두고 실습용 창을 하나 더 연다.
한글 UI에서는 파일 → 새 창이다. Welcome 탭은 x로 닫아도 된다.

2. workspace를 여는 메뉴 선택

새 창에서 File → Open Workspace from File...을 누른다.



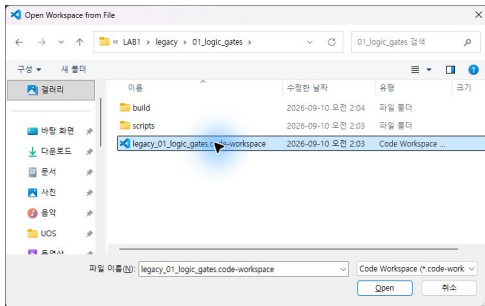
workspace는 소스 폴더·편집 설정·시뮬레이션 작업을 함께 여는 파일이다.

Open File...로 Verilog 파일 하나만 여는 것과 구분한다.

3. 실습 01 workspace 선택

저장소에서 다음 폴더를 찾는다.

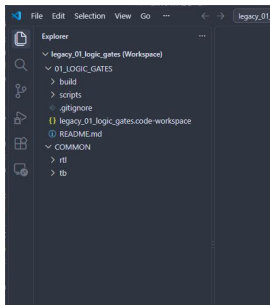
example/fpga_projects_hdl/LAB1/legacy/01_logic_gates



legacy_01_logic_gates.code-workspace를 선택하고
오른쪽 아래 Open을 누른다. PC마다 저장소의 상위 경로는 다르다.

4. 올바른 창인지 확인

왼쪽 Explorer에서 01_LOGIC_GATES와 COMMON을 찾는다.



01_LOGIC_GATES: 실행 작업·결과 폴더. COMMON: RTL·테스트벤치.
목록이 안 보이면 왼쪽 맨 위 파일 모양 아이콘을 누른다.

5. 확장 두 개 확인

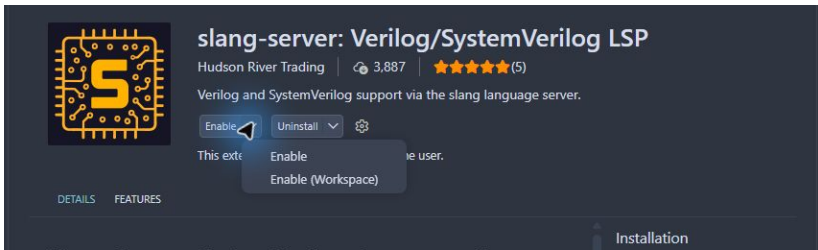
1. 왼쪽 Extensions 아이콘(네모 모양)을 누른다.
단축키는 Ctrl+Shift+X다.
2. 검색창에 @installed slang을 입력한다.
slang-server: Verilog/SystemVerilog LSP를 선택한다.
3. 같은 방법으로 @installed vaporview도 찾는다.

설치되어 있지 않으면 @installed를 빼고 검색한다.
배포자가 각각 Hudson River Trading, Lloyd Ramseyer인지 확인한다.

정확한 ID: hudson-river-trading.vscode-slang
lramseyer.vaporview. 설치 단계는 환경설정 매뉴얼을 따른다.

6. 설치되어 있어도 꺼져 있을 수 있다

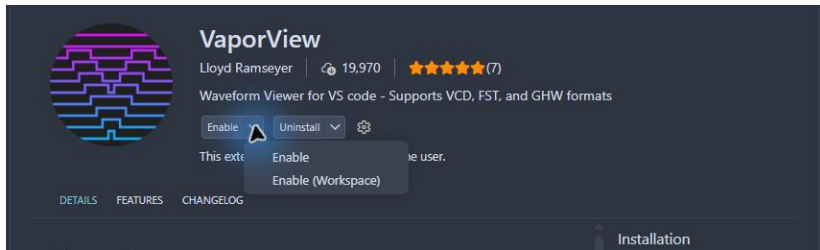
상세 화면에 Enable이 보이면 활성화 상태를 확인한다.



Enable 오른쪽 화살표 → Enable (Workspace)를 선택한다.
이 실습 창에서 활성화한다. 이미 활성화되어 있으면 이 단계는 건너뛰는다.

7. VaporView도 같은 방법으로 확인

@installed vaporview 검색 → 상세 화면 → 활성화 상태 확인.



VaporView는 VCD에 저장된 시간을 파형으로 보여주는 확장이다.
설치·활성화가 끝나면 Explorer 아이콘으로 돌아간다.

8. 실행 전에 진리표를 먼저 쓴다

a	b	x (AND)	y (OR)	z (XOR)
0	0	0	0	0
1	0	0	1	1
0	1	0	1	1
1	1	1	1	0

AND: 둘 다 1이면 1. OR: 하나 이상 1이면 1.

XOR: 두 입력이 서로 다르면 1.

이 실습의 입력 순서는 원본과 같은 $00 \rightarrow 10 \rightarrow 01 \rightarrow 11$ 이다.

9. RTL 파일을 연다

COMMON → rtl → logic_gate.v를 차례로 펼치고 파일을 두 번 누른다.

```
COMMON > rtl > @ logic_gate.v > ...  
1  module logic_gate (  
2      input  wire a,  
3      input  wire b,  
4      output wire x,  
5      output wire y,  
6      output wire z  
7  );  
8      assign x = a & b; // AND  
9      assign y = a | b; // OR  
10     assign z = a ^ b; // XOR  
11 endmodule  
12
```

입력은 a·b, 출력은 x·y·z다. assign은 조합회로의 연결을 나타낸다.
아래 상태 표시줄에 Verilog가 보이는지 확인한다.

10. 테스트벤치에서 회로를 연결한다

COMMON → tb → tb_logic_gate.sv를 두 번 누른다.

```
COMMON > tb > @ tb_logic_gate.sv > ...
1 | timescale 10ns/100ps
2 | module tb_logic_gate;
3 |     reg a, b;
4 |     wire x, y, z;
5 |     integer checked = 0;
6 |     logic_gate dut (.a(a), .b(b), .x(x), .y(y), .z(z));
7 |
8 |     task check(input [2:0] expected);
9 |         begin
10 |             if ({x, y, z} != expected)
11 |                 $fatal(1, "FAIL: a=%b b=%b xyz=%b expected=%b",
12 |                     a, b, {x, y, z}, expected);
13 |             checked = checked + 1;
14 |             $display("PASS: t=%0t a=%b b=%b x=%b y=%b z=%b",
15 |                 $time, a, b, x, y, z);
16 |         end
17 |     endtask
```

dut는 검사할 회로다. check는 $x \cdot y \cdot z$ 를 예상값과 비교한다.
다르면 FAIL로 중단한다. 코드 진단에 빨간 줄이 없다는 것만으로 검사가 끝난 것은 아니다.

11. 입력 자극·저장·종료를 읽는다

테스트벤치를 아래로 내려 initial 부분을 본다.

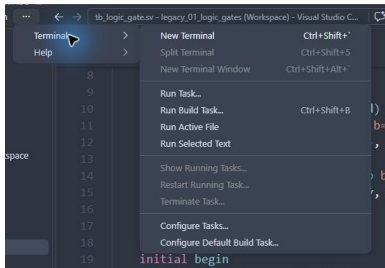
```
18
19     initial begin
20         $timeformat(-9, 0, " ns", 0);
21         $dumpfile("logic_gate.vcd");
22         $dumpvars(0, tb_logic_gate);
23         // Same stimulus order and 200 ns intervals as the legacy manual.
24         a = 0; b = 0; #20; check(3'b000);
25         a = 1; b = 0; #20; check(3'b011);
26         a = 0; b = 1; #20; check(3'b011);
27         a = 1; b = 1; #20; check(3'b110);
28         if (checked != 4) $fatal(1, "FAIL: incomplete test");
29         $display("PASS: all 4 logic-gate input combinations");
30         $finish;
31     end
32 endmodule
33
```

#20은 20ns가 아니라 $20 \times 10\text{ns} = 200\text{ns}$ 다.

VCD를 저장하고 네 조합 검사를 마친 800ns에 종료한다.

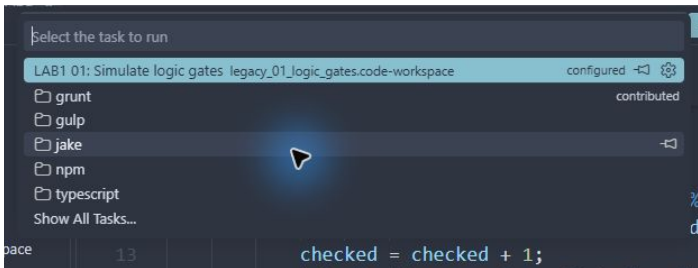
12. 수정했다면 저장부터 한다

1. 코드 탭을 누르고 File → Save(Ctrl+S)를 선택한다. 여러 파일을 고쳤다면 File → Save All을 사용한다.
2. 코드 탭 이름 옆의 미저장 표시가 사라졌는지 확인한다.
3. Terminal → Run Task...를 선택한다. 창이 좁으면 상단 ... 안에 Terminal이 들어 있다.



13. 논리 게이트 시뮬레이션 작업 선택

목록에서 LAB1 01: Simulate logic gates를 누른다.



파일을 따로 지정하지 않아도 작업이 RTL과 테스트벤치를 함께 사용한다. 같은 작업을 Terminal → Run Build Task...(Ctrl+Shift+B)로도 실행할 수 있다.

작업이 없으면 3단계로 돌아가 지정된 workspace를 열었는지 확인한다.

14. 완료 로그를 끝까지 확인

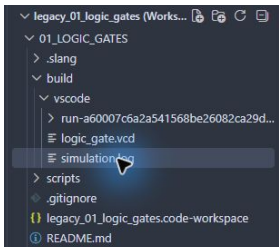
아래 터미널의 작업이 끝날 때까지 기다린다. 결과는 로그 파일에도 저장된다.

```
9
10 source xsim.dir/logic_gate_sim/xsim_script.tcl
11 # xsim {logic_gate_sim} -autoloadwcfg -runall
12 Time resolution is 1 ps
13 run -all
14 PASS: t=200 ns a=0 b=0 x=0 y=0 z=0
15 PASS: t=400 ns a=1 b=0 x=0 y=1 z=1
16 PASS: t=600 ns a=0 b=1 x=0 y=1 z=1
17 PASS: t=800 ns a=1 b=1 x=1 y=1 z=0
18 PASS: all 4 logic-gate input combinations
19 $finish called at time : 800 ns : File "C:/Users/UOS/Documents/ece2_26_2_2/ex
20 exit
21 INFO: [Common 17-206] Exiting xsim at Thu Sep 10 02:04:33 2026...
```

확인할 문구: PASS: all 4 logic-gate input combinations.
200·400·600·800ns의 PASS 네 줄과 800ns 종료를 함께 확인한다.

15. 로그와 파형이 생긴 위치

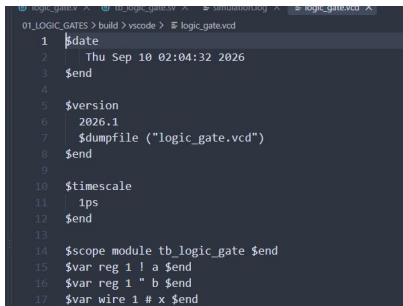
1. Explorer에서 01_LOGIC_GATES → build → vscode를 펼친다.
2. simulation.log를 두 번 누르면 방금 실행한 전체 로그를 읽을 수 있다.
3. logic_gate.vcd가 파형 파일이다. 다음 단계에서 연다.



run-... 폴더에는 컴파일·연결·실행 로그가 따로 남는다.
실패했다면 예전 캡처 대신 이번 실행의 오류와 원인을 확인한다.

16. VCD가 글자로 열리면

파일은 정상이어도 Text Editor로 열면 파형 대신 내용이 보인다.

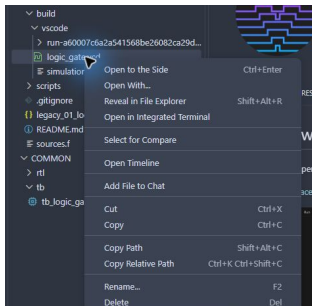


```
01_LOGIC_GATES > build > vscode > logic_gate.vcd
1 $date
2   Thu Sep 10 02:04:32 2026
3 $end
4
5 $version
6   2026.1
7   $dumpfile ("logic_gate.vcd")
8 $end
9
10 $timescale
11   1ps
12 $end
13
14 $scope module tb_logic_gate $end
15 $var reg 1 l a $end
16 $var reg 1 " b $end
17 $var wire 1 # x $end
```

이 화면에서 값을 직접 고치지 않는다. 다음 단계처럼 VaporView로 다시 연다.

17. Open With... 선택

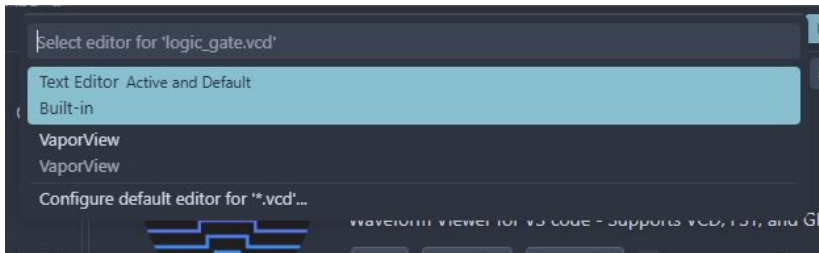
Explorer의 `logic_gate.vcd`를 마우스 오른쪽 버튼으로 누른다.



메뉴에서 `Open With...`를 누른다.
`Open in Integrated Terminal`과 구분한다.

18. VaporView로 연다

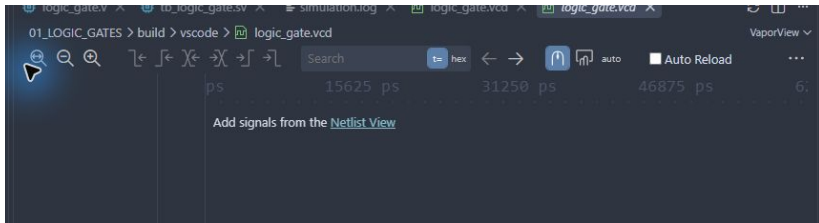
편집기 선택 목록에서 VaporView를 누른다.



VaporView가 목록에 없으면 5-7단계에서 설치·활성화를 다시 확인한다.
파일 탭에 파형 아이콘과 VaporView 표시가 나타나는지 확인한다.

19. 빈 파형 화면에서 신호 목록 열기

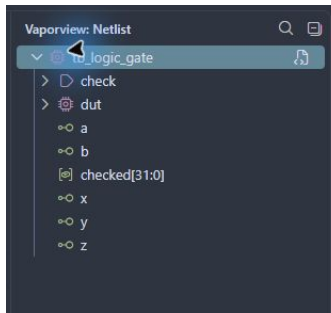
처음에는 신호가 하나도 없을 수 있다. 화면 안의 Netlist View를 누른다.



왼쪽에 Vaporview: Netlist가 나타난다.
tb_logic_gate 왼쪽의 삼각형을 눌러 계층을 펼친다.

20. $a \cdot b \cdot x \cdot y \cdot z$ 를 추가한다

a, b, x, y, z를 차례로 두 번 누른다. 오른쪽 파형 목록에 한 줄씩 생긴다.

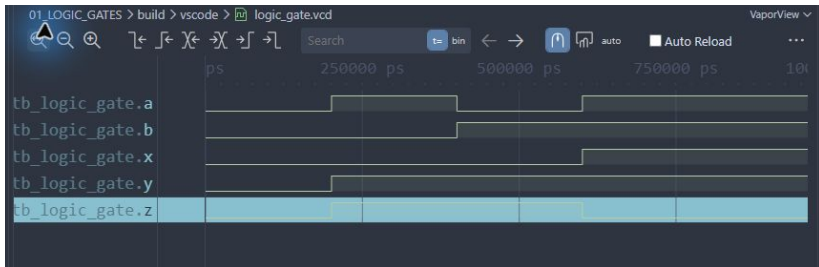


checked는 검사 횟수이며 회로 출력이 아니다. 이번 파형에는 넣지 않는다.

dut 아래에도 같은 회로 신호가 있으므로 중복해서 추가하지 않는다.

21. 파형이 평평하면 시간 범위를 확인

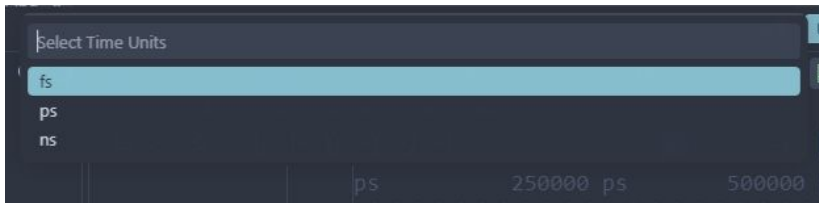
파형 위 도구 모음의 맨 왼쪽 돋보기(Zoom to Fit)를 누른다.



확대된 시작 구간만 보다가 전체 구간을 보게 되면 입력 변화가 나타난다.
화면의 200000 ps는 200ns와 같다. 다음 단계에서 단위를 바꾼다.

22. 시간 단위를 ns로 표시

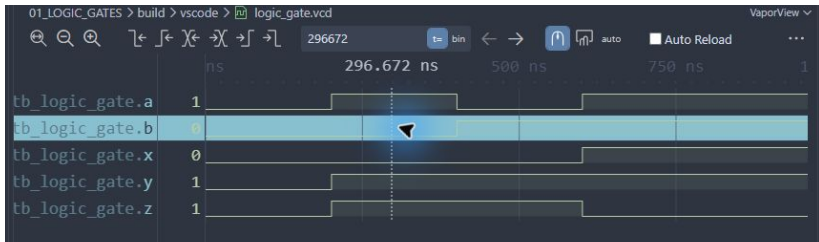
오른쪽 아래 상태 표시줄의 Time Units: ps를 누르고 ns를 선택한다.



파형 데이터 자체를 바꾸는 것이 아니라 표시 단위만 바꾼다.
로그의 시간과 파형의 시간을 같은 단위로 비교한다.

23. 입력 10 구간의 값을 읽는다

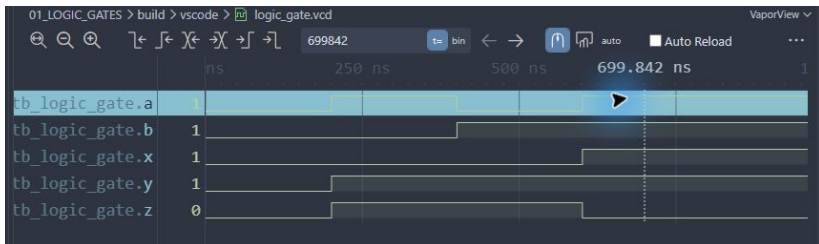
200~400ns 사이 파형을 한 번 누른다. 세로 커서와 신호별 값이 나타난다.



실제 캡처의 커서는 약 296.7ns다. a=1, b=0, x=0, y=1, z=1.
전환 경계 대신 구간 안쪽을 읽는다. 다른 세 구간도 같은 방법으로 확인한다.

24. 입력 11에서는 XOR가 0이다

600–800ns 사이를 눌러 마지막 입력 조합을 확인한다.



약 699.8ns에서 $a=1$, $b=1$, $x=1$, $y=1$, $z=0$.

OR와 XOR의 차이가 드러나는 구간이다. 이 차이를 레포트에서 설명한다.

25. 파형과 진리표를 한 줄씩 비교

시간 구간(ns)	a	b	x	y	z
0-200	0	0	0	0	0
200-400	1	0	0	1	1
400-600	0	1	0	1	1
600-800	1	1	1	1	0

로그는 각 구간의 끝에서 값을 검사하고, 파형은 구간 전체의 변화를 보여준다.

원본과 같은 입력 순서이며 800ns 이후로 테스트가 더 진행된 것은 아니다.

화면 맞춤 때문에 종료 시각 바깥까지 축이 보일 수 있다.

26. 실험 전 레포트에 기록할 내용

1. 회로 목적, $a \cdot b$ 입력과 $x \cdot y \cdot z$ 출력, 실행 전에 작성한 진리표.
2. RTL의 AND·OR·XOR 연결과 테스트벤치의 네 입력 조합.
3. 사용한 workspace·시뮬레이터 버전·작업 이름, 시간 단위와 200ns 간격·800ns 종료 이유.
4. PASS 네 줄과 최종 완료 문구를 포함한 자신의 실행 로그 캡처.
5. $a \cdot b \cdot x \cdot y \cdot z$, 시간축, 커서 값이 보이는 자신의 파형 캡처.
6. 네 구간별 예상값·실제값 비교, OR와 XOR가 달라지는 이유, 오류가 있었다면 수정·재실행 과정.

이 자료의 캡처를 자신의 실험 결과로 제출하지 않는다.
코드를 저장소에서 가져와도 구조와 결과를 설명할 수 있어야 한다.

27. 수정 후에는 다시 실행한다

1. RTL 또는 테스트벤치를 수정했다면 Save / Save All.
2. Run Task... → LAB1 01: Simulate logic gates 재실행.
3. 이번 실행의 PASS·종료 시각과 로그를 다시 확인.
4. VaporView 탭 오른쪽 위의 새로고침(원형 화살표)을 누르거나 VCD를 다시 연다.
5. 동일한 신호·시간 구간을 비교하고 새 캡처를 레포트에 넣는다.

소스만 고치고 이전 파형을 보면 수정 결과를 확인할 수 없다.
실행 실패 후에는 이전 파형을 성공 결과로 사용하지 않는다.

28. 막혔을 때 돌아갈 곳

보이는 문제	확인할 것
시뮬레이션 작업 없음 도구를 찾을 수 없음	지정한 .code-workspace를 열었는지(3단계). Vivado bin의 PATH 또는 VIVADO_BIN 설정 후 VS Code 다시 열기.
코드가 Plain Text 모듈을 찾지 못하는 진단	slang 확장 활성화·파일 확장자(5-6단계). sources.f와 .slang 설정 확인, 명령 팔레트의 slang: Restart Language Server.
VCD가 글자로 보임 파형 없음·평평한 파형 FAIL 또는 컴파일 오류	Open With... → VaporView(16-18단계). PASS 확인 → 신호 추가 → Zoom to Fit. 이번 실행의 로그·소스 수정 → 저장 → 재실행.

29. 이제 Vivado GUI로 넘어간다

- 네 입력 조합의 PASS, 800ns 종료, 파형·진리표의 일치를 확인했다.
- 실험 전 레포트에 실행 방법·코드 설명·실제 로그·파형 해석을 기록했다.
- 다음은 레거시 원본의 Vivado 절차다. 프로젝트를 만들고 같은 회로·입력을 Vivado에서도 비교한다.
- 실험 후에는 실제 FPGA에 비트스트림을 기록하고 동작 사진·영상·GitHub 링크를 실험 후 레포트에 정리한다.

▶ 레거시 Vivado 실습 시작 Vivado 세부 목차

공식 참고: VS Code workspace · Tasks · VaporView

레거시 01 · 논리 게이트

[실습 1] AND, OR, XOR 게이트 구현

- 입력 a, b에 대한 출력 x, y, z를 확인한다.
- 원본의 프로젝트 이름: logic_gate.
- 대상 부품: Spartan-7 xc7s75fpga484-1.

실습 순서

VS Code 사전 시뮬레이션·실험 전 레포트
Vivado 실습·FPGA 기록·동작 확인
사진·영상·GitHub·실험 후 레포트

작성일 2026. 09. 10. 이해리

원자료: 박동욱 교수, 서울시립대학교, 「논리 게이트 구현」.

다음 원문 페이지는 2022년 보관 PDF의 사진과 설명을 재구성했다.

원본: 논리 게이트 구현, p.3 레거시 01



서울시립대학교
UNIVERSITY OF SEOUL

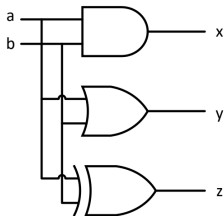
레거시 01 · Vivado 목차

설계 기획	블록도·진리표
(1) 프로젝트 선언	부품·경로·프로젝트 설정
(2) 로직 설계	원본 코드·소스 등록
(3) Assignment	입출력 핀·전기 규격
(4) Compile	합성·배치배선
(5) Simulation	테스트벤치·파형
(6) Programming	비트스트림·실제 장치 기록
(7) 동작 확인	스위치 입력·LED 출력

원본 순서를 보존했다. 수업에서는 합성·구현에 앞서 VS Code와 Vivado에서 시뮬레이션한다. 원문 보충 설명 · 실험 전·후 레포트 · 자료 안내

설계 기획

블록도 및 진리표



서울시립대학교 전자전기컴퓨터공학부

입력		출력		
a	b	AND x	OR y	XOR z
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

원본: 논리 게이트 구현, p.4 레거시 01

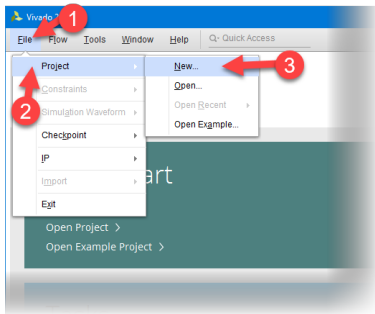


서울시립대학교
UNIVERSITY OF SEOUL

로직 설계 및 합성

(1) 프로젝트 선언

- Vivado 실행
- File > Project > New



원본: 논리 게이트 구현, p.5 레거시 01

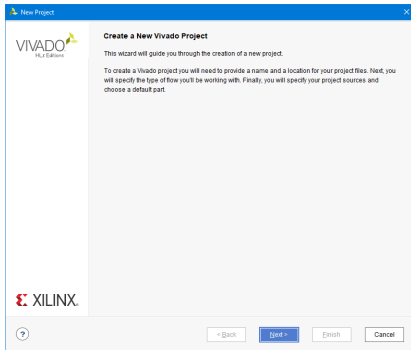


서울시립대학교
UNIVERSITY OF SEOUL

로직 설계 및 합성

(1) 프로젝트 선언

- Next



원본: 논리 게이트 구현, p.6 레거시 01

로직 설계 및 합성

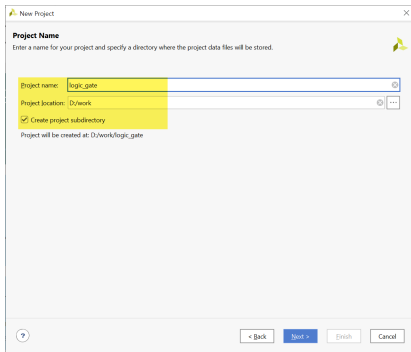
(1) 프로젝트 선언

- Project Name과 Project Location 설정
- Project Name : `logic_gate`
- Project location : `d:/work`
- Create project subdirectory : 체크
- `d:/work` 폴더에 `logic_gate` 폴더를 생성

로직 설계 및 합성

(1) 프로젝트 선언

원문 설명에 대응하는 실제 화면



A screenshot of a 'New Project' dialog box. The dialog has a title bar with a yellow triangle icon and a close button. The main content area is titled 'Project Name' and contains the instruction 'Enter a name for your project and specify a directory where the project data files will be stored.' Below this, there are two input fields: 'Project name:' with the text 'logic_gate' and 'Project location:' with the text 'D:/work'. To the right of the 'Project location:' field is a button with three dots. Below these fields is a checkbox labeled 'Create project subdirectory' which is checked. Underneath the checkbox, it says 'Project will be created at: D:/work/logic_gate'. At the bottom of the dialog, there are four buttons: a question mark icon, '< Back', 'Next >', and 'Cancel'.

원본: 논리 게이트 구현, p.7 레거시 01

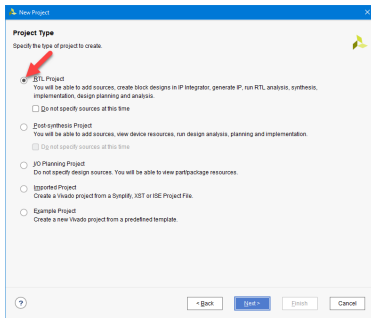


서울시립대학교
UNIVERSITY OF SEOUL

로직 설계 및 합성

(1) 프로젝트 선언

- Project Type 설정
- RTL Project



원본: 논리 게이트 구현, p.8 레거시 01

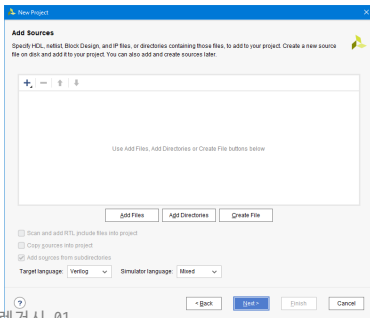


서울시립대학교
UNIVERSITY OF SEOUL

로직 설계 및 합성

(1) 프로젝트 선언

- Add source와 Add Constraints 설정
- 설계된 로직 파일과 설정된 핀 정보 파일이 없기 때문에 Next 버튼 누름.



원본: 논리 게이트 구현, p.9 레거시 01

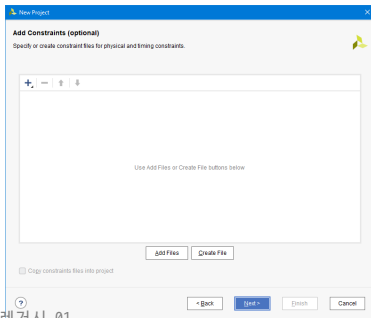


서울시립대학교
UNIVERSITY OF SEOUL

로직 설계 및 합성

(1) 프로젝트 선언

- Add source와 Add Constraints 설정
- 설계된 로직 파일과 설정된 핀 정보 파일이 없기 때문에 Next 버튼 누름.



원본: 논리 게이트 구현, p.9 레거시 01

로직 설계 및 합성

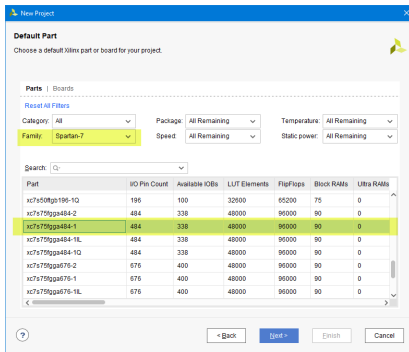
(1) 프로젝트 선언

- Device 설정
- Family : Spartan-7
- Part : xc7s75fgga484-1

로직 설계 및 합성

(1) 프로젝트 선언

원문 설명에 대응하는 실제 화면



원본: 논리 게이트 구현, p.10 레거시 01

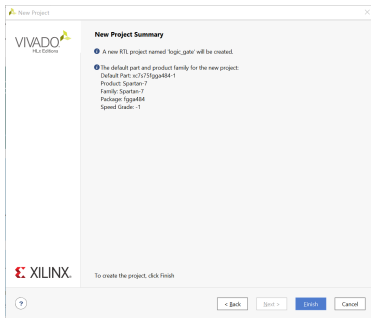


서울시립대학교
UNIVERSITY OF SEOUL

로직 설계 및 합성

(1) 프로젝트 선언

- 설정된 Summary 확인.
- Finish를 눌러 프로젝트 선언 완료

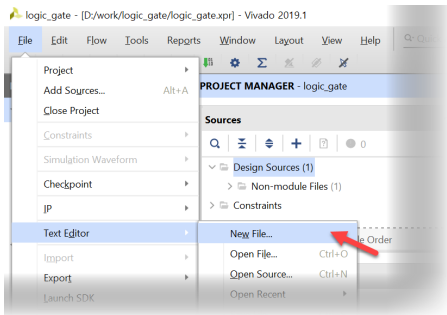


원본: 논리 게이트 구현, p.11 레거시 01

로직 설계 및 합성

(2) 로직 설계

- VIVADO 프로그램
- Text Editor > New File.. 선택



원본: 논리 게이트 구현, p.12 레거시 01

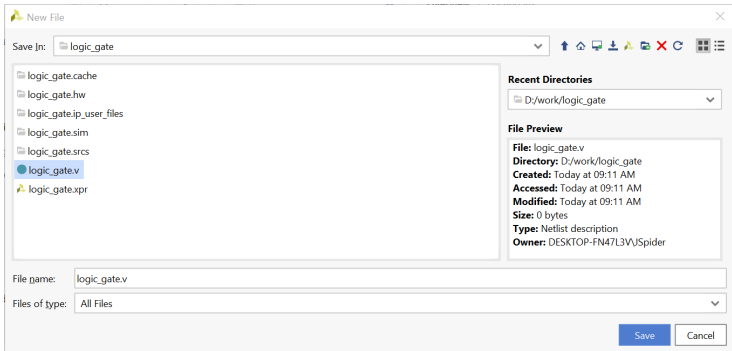


서울시립대학교
UNIVERSITY OF SEOUL

로직 설계 및 합성

(2) 로직 설계

- logic_gate.v 파일로 저장



원본: 논리 게이트 구현, p.13 레거시 01

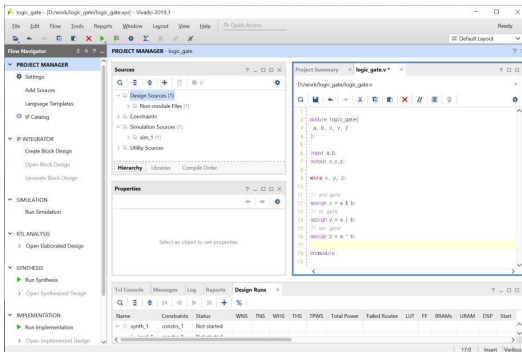


서울시립대학교
UNIVERSITY OF SEOUL

로직 설계 및 합성

(2) 로직 설계

- 로직 설계



원본: 논리 게이트 구현, p.14 레거시 01



서울시립대학교
UNIVERSITY OF SEOUL

로직 설계 및 합성

(2) 로직 설계

- 로직 설계

```
1 |  
2 | module logic_gate(  
3 |     a, b, x, y, z  
4 | );  
5 |  
6 | input a,b;  
7 | output x,y,z;  
8 |  
9 | wire x, y, z;  
10 |  
11 | // and gate  
12 | assign x = a & b;  
13 | // or gate  
14 | assign y = a | b;  
15 | // xor gate  
16 | assign z = a ^ b;  
17 |  
18 | endmodule  
19 |
```

원본: 논리 게이트 구현, p.14 레거시 01

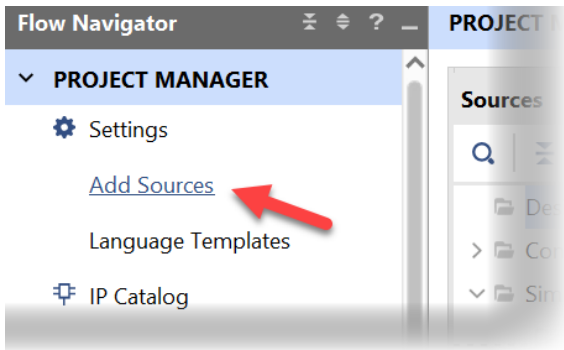


서울시립대학교
UNIVERSITY OF SEOUL

로직 설계 및 합성

(2) 로직 설계

- PROJECT MANAGER > Add Sources 선택



원본: 논리 게이트 구현, p.15 레거시 01

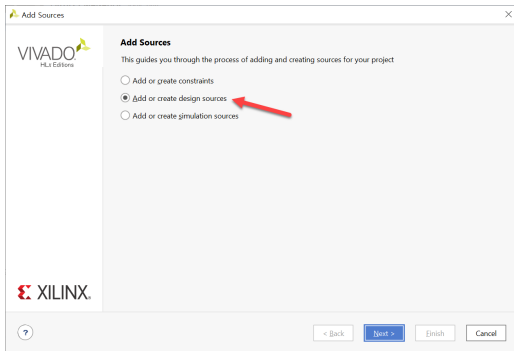


서울시립대학교
UNIVERSITY OF SEOUL

로직 설계 및 합성

(2) 로직 설계

- Add Sources 창에서 Add or Create design sources 선택

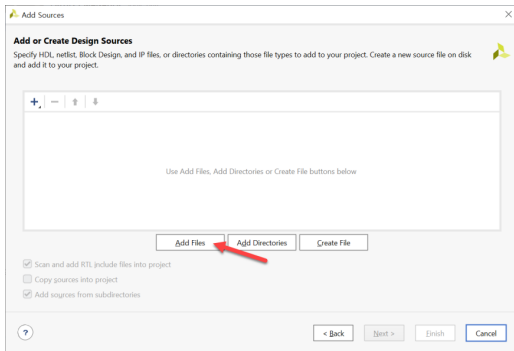


원본: 논리 게이트 구현, p.16 레거시 01

로직 설계 및 합성

(2) 로직 설계

- Add Files 버튼을 눌러, 앞의 logic_gate.v 파일 추가



원본: 논리 게이트 구현, p.17 레거시 01

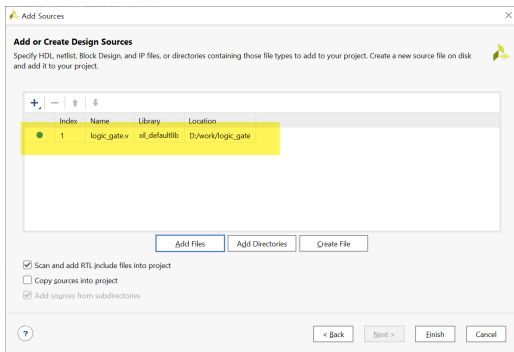


서울시립대학교
UNIVERSITY OF SEOUL

로직 설계 및 합성

(2) 로직 설계

- Add Files 버튼을 눌러, 앞의 logic_gate.v 파일 추가



원본: 논리 게이트 구현, p.17 레거시 01

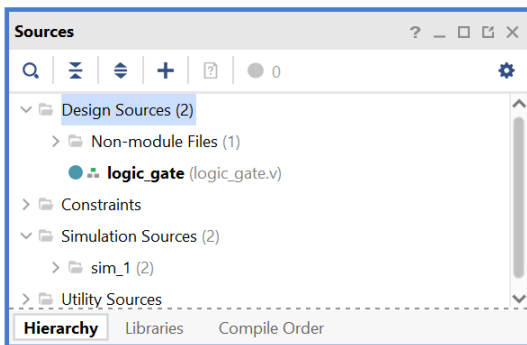


서울시립대학교
UNIVERSITY OF SEOUL

로직 설계 및 합성

(2) 로직 설계

- Sources 창에서 파일 추가된 것을 확인



원본: 논리 게이트 구현, p.18 레거시 01



서울시립대학교
UNIVERSITY OF SEOUL

로직 설계 및 합성

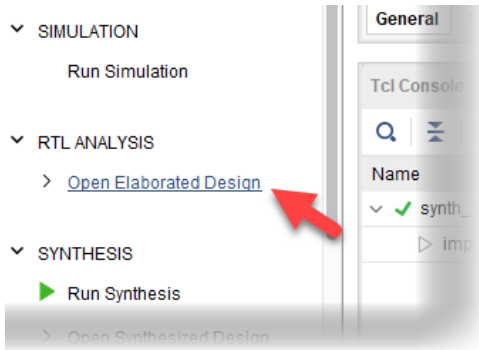
(3) Assignment

- 프로젝트의 타겟이 되는 FPGA 디바이스와 설계된 로직에서 사용하는 입출력 단자를 하드웨어와 연결하기 위하여 핀을 설정하는 것

로직 설계 및 합성

(3) Assignment

- PROJECT MANAGER의 RTL ANALYSIS 탭의 Open Elaborated Design 부분을 마우스로 클릭



원본: 논리 게이트 구현, p.20 레거시 01



서울시립대학교
UNIVERSITY OF SEOUL

로직 설계 및 합성

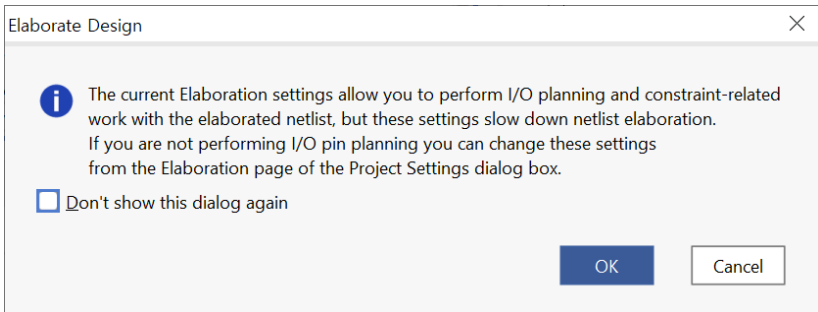
(3) Assignment

- 메시지 창이 나타난다.
- OK 버튼을 누르면 Elaboration 과정이 진행됨.
- 문법에 오류부분도 같이 체크하여 오류가 있다면 에러 메시지가 출력됨.

로직 설계 및 합성

(3) Assignment

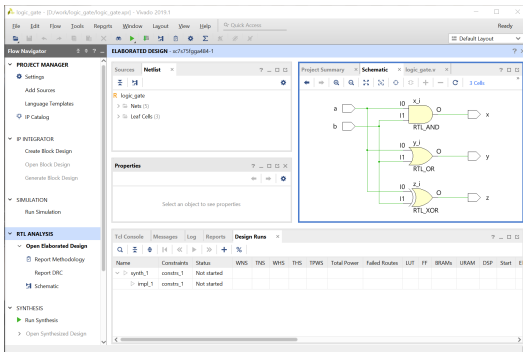
원문 설명에 대응하는 실제 화면



로직 설계 및 합성

(3) Assignment

- 설계된 Verilog HDL 구문이 Schematic으로 표현되는 것을 확인



원본: 논리 게이트 구현, p.22 레거시 01

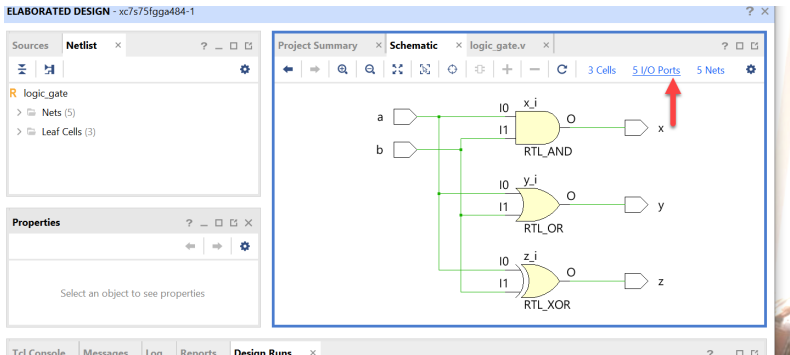


서울시립대학교
UNIVERSITY OF SEOUL

로직 설계 및 합성

(3) Assignment

- Schematic 화면 오른 쪽 윗 부분의 5 I/O Ports 선택



원본: 논리 게이트 구현, p.23 레거시 01



서울시립대학교
UNIVERSITY OF SEOUL

로직 설계 및 합성

(3) Assignment

- I/O Std를 “LVCMOSS33” 으로 변경
- 입/출력 포트의 Voltage Level을 3.3V로 설정하는 것.

포트 이름 핀 번호			포트 이름 핀 번호		
a	K4	SW_1	x	L4	LED1
b	N8	SW_2	y	M4	LED2
			z	M2	LED3

원문 표기 유지. 실제 속성값은 **보충 설명**을 확인한다.

로직 설계 및 합성

(3) Assignment

- Package Pin 부분에 핀 설정
- I/O Std를 “LVCMOSS33” 으로 변경
- 입/출력 포트의 Voltage Level을 3.3V로 설정하는 것.



로직 설계 및 합성

(3) Assignment

원문 설명에 대응하는 실제 화면

Tcl Console Messages Log Reports Design Runs Find Results x											
Q [Icons]											
Name	Direction	Interface	Neg Diff Pair	Package Pin	Fixed	Bank	I/O Std	Vcco	Vref	Drive Strength	
a	IN			K4	☒	35	LVC MOS33*	3.300			
b	IN			N8	☒	35	LVC MOS33*	3.300			
x	OUT			L4	☒	35	LVC MOS33*	3.300		12	
y	OUT			M4	☒	35	LVC MOS33*	3.300		12	
z	OUT			M2	☒	35	LVC MOS33*	3.300		12	

원본: 논리 게이트 구현, p.25 레거시 01

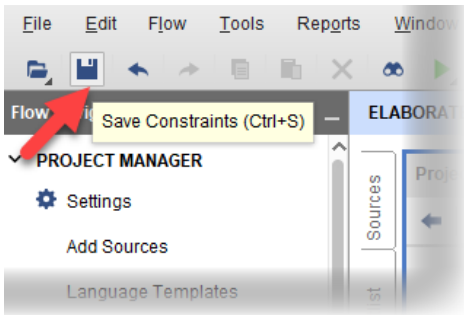


서울시립대학교
UNIVERSITY OF SEOUL

로직 설계 및 합성

(3) Assignment

- 핀 설정 저장
- 파일 명은 logic_gate로 설정



원본: 논리 게이트 구현, p.26 레거시 01

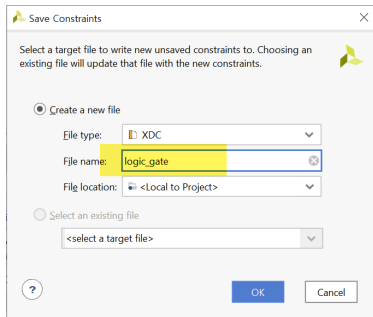


서울시립대학교
UNIVERSITY OF SEOUL

로직 설계 및 합성

(3) Assignment

- 핀 설정 저장
- 파일명은 logic_gate로 설정



원본: 논리 게이트 구현, p.26 레거시 01

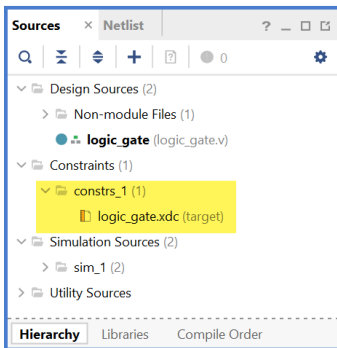


서울시립대학교
UNIVERSITY OF SEOUL

로직 설계 및 합성

(3) Assignment

- Sources 창에 추가된 것 확인



원본: 논리 게이트 구현, p.27 레거시 01

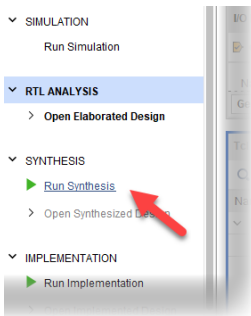


서울시립대학교
UNIVERSITY OF SEOUL

로직 설계 및 합성

(4) Compile

- SYNTHESIS > Run Synthesis
- 설계된 Verilog HDL 코드를 합성하여 최적화 시키는 과정



원본: 논리 게이트 구현, p.28 레거시 01

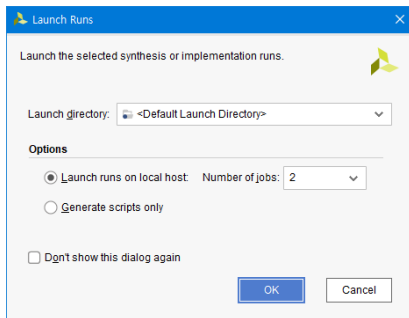


서울시립대학교
UNIVERSITY OF SEOUL

로직 설계 및 합성

(4) Compile

- SYNTHESIS > Run Synthesis
- 설계된 Verilog HDL 코드를 합성하여 최적화 시키는 과정



원본: 논리 게이트 구현, p.28 레거시 01

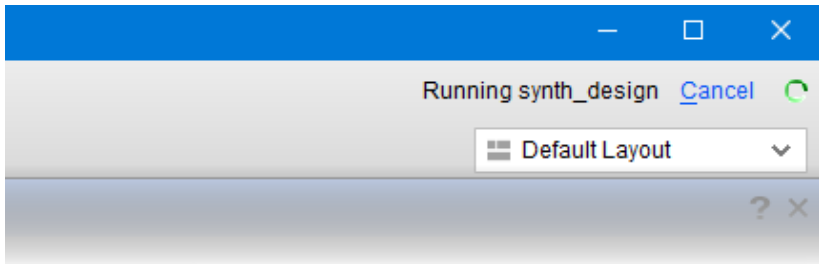


서울시립대학교
UNIVERSITY OF SEOUL

로직 설계 및 합성

(4) Compile

- Synthesis 동작 확인 위치
- 프로그램 상단 오른쪽 회전



로직 설계 및 합성

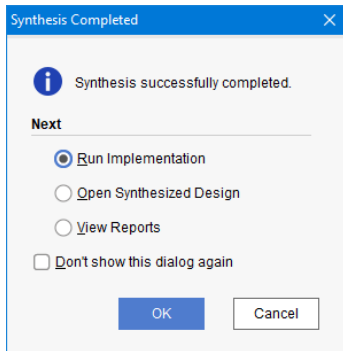
(4) Compile

- Synthesis 후 IMPLEMENTATION 진행
- Implementation
- 합성한 결과를 이용하여 실제 FPGA의 Logic Cell 단위로 바꾸어 FPGA내에 위치시키고 연결하는 Place & Route를 하는 과정

로직 설계 및 합성

(4) Compile

원문 설명에 대응하는 실제 화면



원본: 논리 게이트 구현, p.30 레거시 01

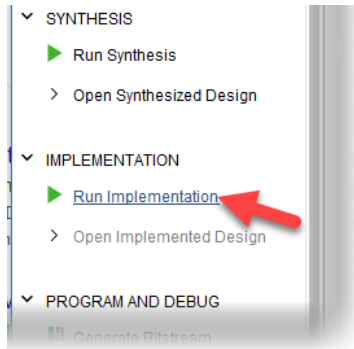


서울시립대학교
UNIVERSITY OF SEOUL

로직 설계 및 합성

(4) Compile

원문 설명에 대응하는 실제 화면



원본: 논리 게이트 구현, p.30 레거시 01

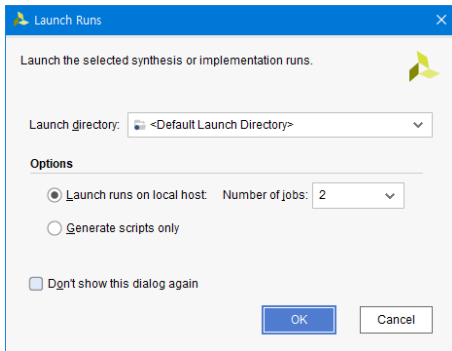


서울시립대학교
UNIVERSITY OF SEOUL

로직 설계 및 합성

(4) Compile

- Synthesis 후 IMPLEMENTATION 진행



원본: 논리 게이트 구현, p.31 레거시 01

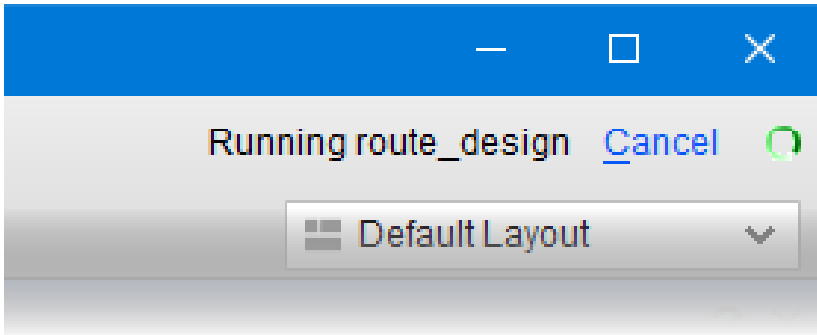


서울시립대학교
UNIVERSITY OF SEOUL

로직 설계 및 합성

(4) Compile

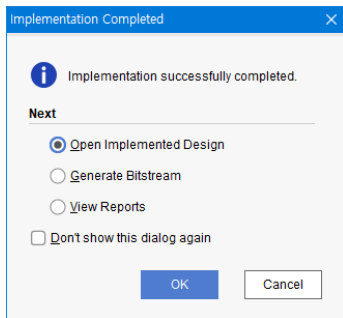
- Synthesis 후 IMPLEMENTATION 진행



로직 설계 및 합성

(4) Compile

- Implementation 이 끝난 화면
- Open Implemented Design 선택 후 OK



원본: 논리 게이트 구현, p.32 레거시 01

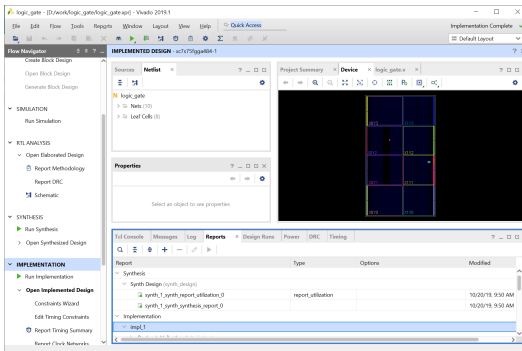


서울시립대학교
UNIVERSITY OF SEOUL

로직 설계 및 합성

(4) Compile

- Implementation 결과 확인



원본: 논리 게이트 구현, p.33 레거시 01



서울시립대학교
UNIVERSITY OF SEOUL

로직 설계 및 합성

(5) Simulation

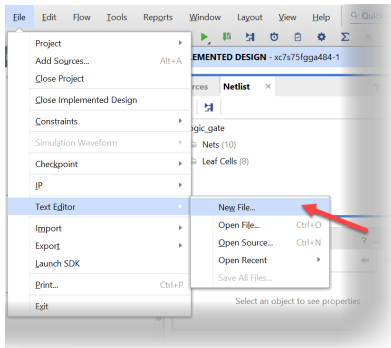
- 소프트웨어적으로 검증하는 과정
- 테스트 벤치 파일이 필요함.
- 시뮬레이션에 대한 입력 조건을 설정한 파일



로직 설계 및 합성

(5) Simulation

- File > Text Editor > New File 선택



원본: 논리 게이트 구현, p.35 레거시 01



서울시립대학교
UNIVERSITY OF SEOUL

로직 설계 및 합성

(5) Simulation

- testbench.v 파일로 저장

```
1  `timescale 10ns / 100ps
2
3  module testbench();
4  // input
5  reg a, b;
6  // output
7  wire x, y, z;
8  // Instantiate the U1
9  logic_gate u1(a, b, x, y, z);
10 // Specify input stimulus
11 initial begin
12     a = 0; b = 0;
13
14     #20 a = 1;
15     #20 a = 0; b = 1;
16     #20 a = 1;
17 end
18
19 endmodule
20
```

원본: 논리 게이트 구현, p.36 레거시 01

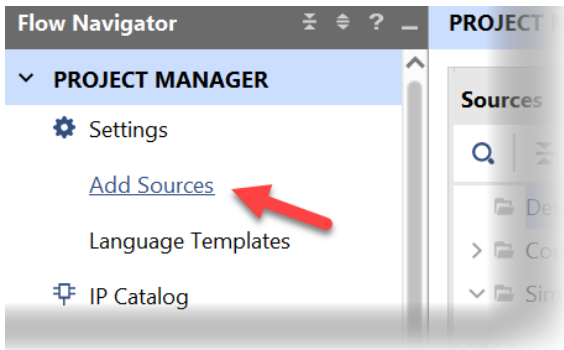


서울시립대학교
UNIVERSITY OF SEOUL

로직 설계 및 합성

(5) Simulation

- PROJECT MANAGER Add Source 선택



원본: 논리 게이트 구현, p.37 레거시 01

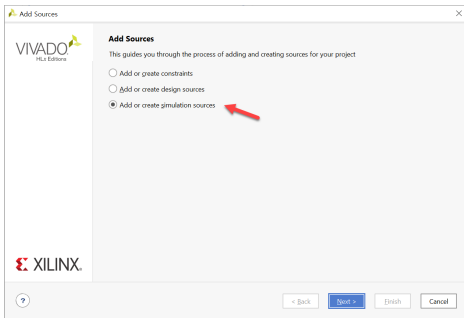


서울시립대학교
UNIVERSITY OF SEOUL

로직 설계 및 합성

(5) Simulation

- [Add or create simulation source] 선택
- Next

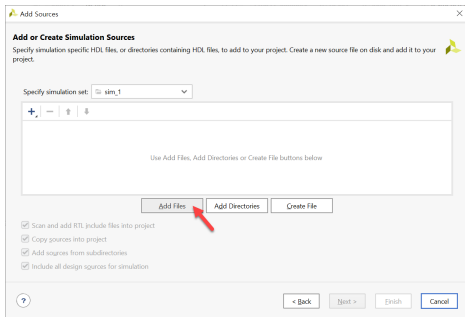


원본: 논리 게이트 구현, p.38 레거시 01

로직 설계 및 합성

(5) Simulation

- [Add Files]을 선택
- testbench.v 파일 선택



원본: 논리 게이트 구현, p.39 레거시 01

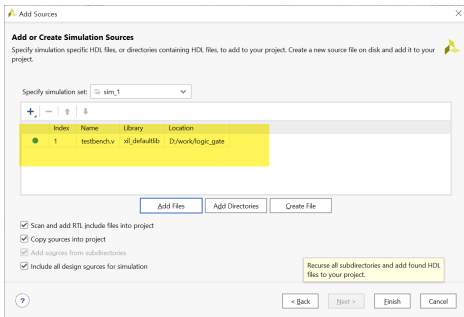


서울시립대학교
UNIVERSITY OF SEOUL

로직 설계 및 합성

(5) Simulation

- [Add Files]을 선택
- testbench.v 파일 선택



원본: 논리 게이트 구현, p.39 레거시 01

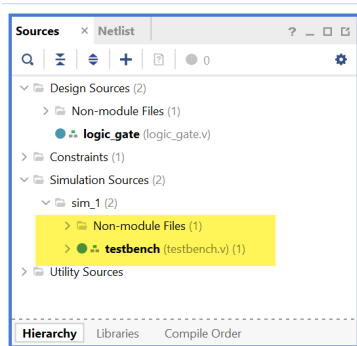


서울시립대학교
UNIVERSITY OF SEOUL

로직 설계 및 합성

(5) Simulation

- Simulation Sources 파일 추가 확인



원본: 논리 게이트 구현, p.40 레거시 01

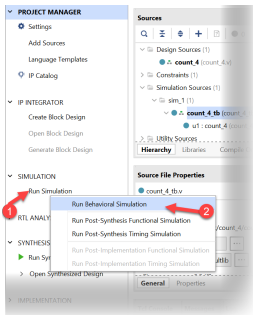


서울시립대학교
UNIVERSITY OF SEOUL

로직 설계 및 합성

(5) Simulation

- SIMULATION > Run Simulation
- Run Behavioral Simulation 실행



원본: 논리 게이트 구현, p.41 레거시 01

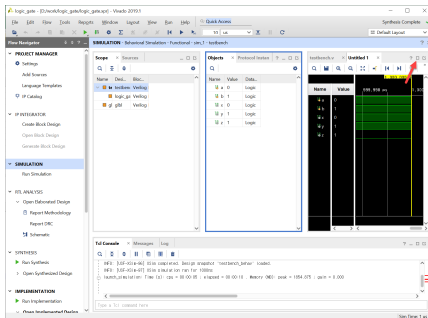


서울시립대학교
UNIVERSITY OF SEOUL

로직 설계 및 합성

(5) Simulation

- 결과 확인
- 상단 우측의 Maximize 버튼 클릭



원본: 논리 게이트 구현, p.42 레거시 01

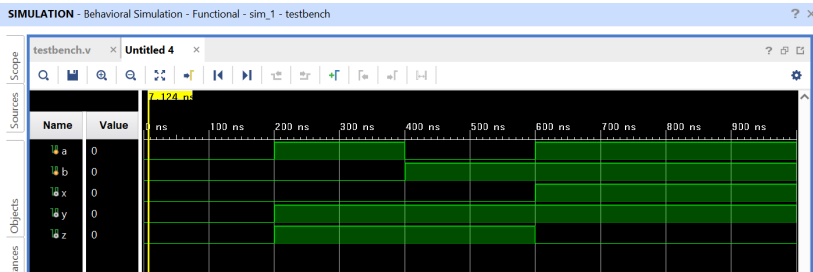


서울시립대학교
UNIVERSITY OF SEOUL

로직 설계 및 합성

(5) Simulation

- Zoom Fit 를 눌러 결과 확인



원본: 논리 게이트 구현, p.43 레거시 01

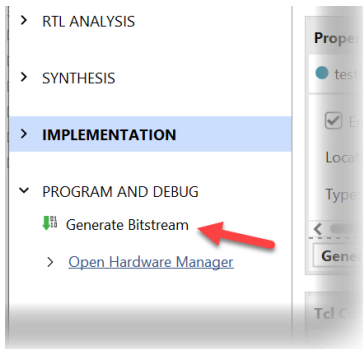


서울시립대학교
UNIVERSITY OF SEOUL

로직 설계 및 합성

(6) Programming

- Project Manager 창 PROGRAM AND DEBUG > Generate Bitstream



원본: 논리 게이트 구현, p.44 레거시 01

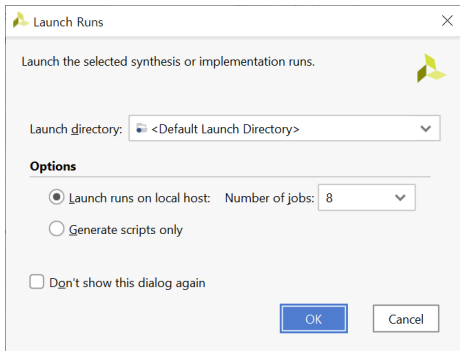


서울시립대학교
UNIVERSITY OF SEOUL

로직 설계 및 합성

(6) Programming

- Project Manager 창 PROGRAM AND DEBUG > Generate Bitstream



원본: 논리 게이트 구현, p.44 레거시 01

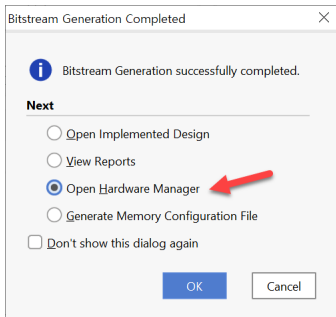


서울시립대학교
UNIVERSITY OF SEOUL

로직 설계 및 합성

(6) Programming

- 프로그래밍 파일 생성 완료 후
- Open Hardware Manager



원본: 논리 게이트 구현, p.45 레거시 01

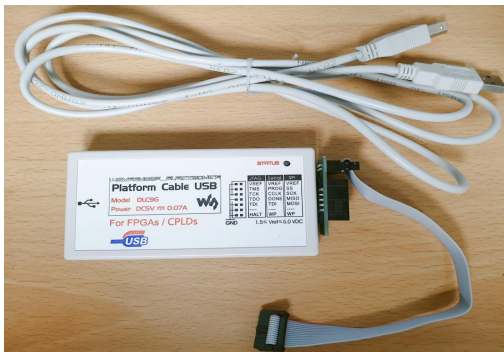


서울시립대학교
UNIVERSITY OF SEOUL

로직 설계 및 합성

(6) Programming

- 하드웨어 준비



원본: 논리 게이트 구현, p.46 레거시 01



서울시립대학교
UNIVERSITY OF SEOUL

로직 설계 및 합성

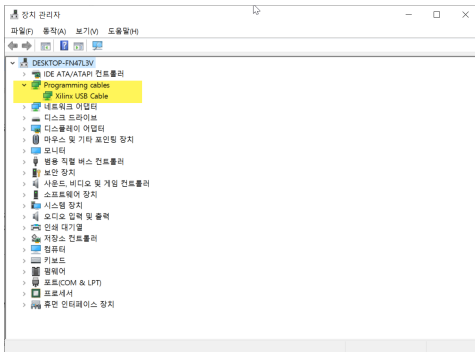
(6) Programming

- 장비의 전원이 꺼져 있는지 확인
- Xilinx Platform Cable에 USB Cable의 한 쪽 끝을 연결
- 반대쪽 끝은 PC의 USB 포트에 연결
- Xilinx Platform Cable에 연결된 2x14핀의 플랫 케이블은 장비의 Xilinx 모듈의 JTAG 포트에 연결
- 장비의 전원을 켜다.

로직 설계 및 합성

(6) Programming

- Xilinx Platform II Cable을 PC에 연결한 후, PC의 장치관리자에서 장치 인식 확인



원본: 논리 게이트 구현, p.48 레거시 01

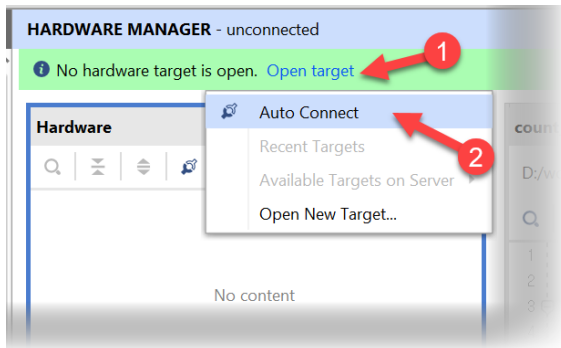


서울시립대학교
UNIVERSITY OF SEOUL

로직 설계 및 합성

(6) Programming

- Open Target > Auto Connect



원본: 논리 게이트 구현, p.49 레거시 01

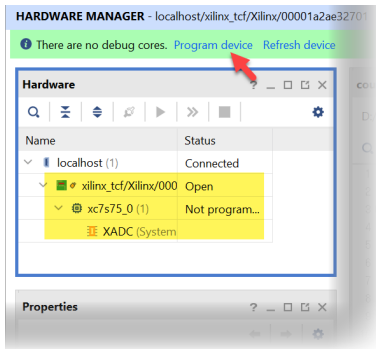


서울시립대학교
UNIVERSITY OF SEOUL

로직 설계 및 합성

(6) Programming

- Program Device 선택



원본: 논리 게이트 구현, p.50 레거시 01



서울시립대학교
UNIVERSITY OF SEOUL

로직 설계 및 합성

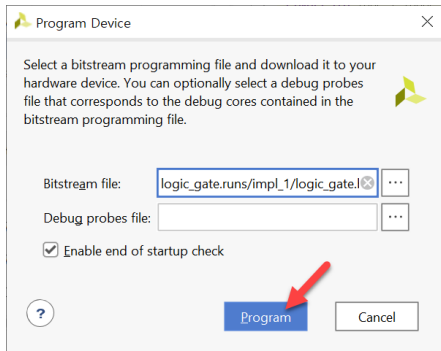
(6) Programming

- Bitstream File 선택
- <Project Folder> \ logic_gate.runs \ impl_1 \ logic_gate.bit 파일 선택
- Program 버튼 클릭

로직 설계 및 합성

(6) Programming

원문 설명에 대응하는 실제 화면



원본: 논리 게이트 구현, p.51 레거시 01



서울시립대학교
UNIVERSITY OF SEOUL

로직 설계 및 합성

(7) 동작 확인

- SW 1와 SW 2를 눌러 장비에서 어떻게 동작하는지 확인.

보충 · 원문 표기와 실행 조건

- 원본 p.24-25의 LVCM0SS33은 오타다.
실제 I/O 표준은 LVCM0S33을 선택한다.
- d:/work는 원본의 예시 작업 경로다.
실제 프로젝트를 저장한 경로를 사용한다.
- 원본 테스트벤치는 10 ns 단위에서 #20마다
입력을 바꾼다. 입력 전환 간격은 200 ns다.
- 원본의 a,b 입력 순서는 00, 10, 01, 11이다.
출력 x,y,z를 AND·OR·XOR 진리표와 비교한다.

이 페이지는 원본과 구분한 보충 설명이다. 원본 코드 이미지는 유지했다.

보충 · VS Code 사전 시뮬레이션

1. logic_gate.v와 테스트벤치를 VS Code에서 연다.
2. 선택한 시뮬레이터로 네 입력 조합을 모두 실행한다.
3. 원본 테스트벤치에는 자동 종료·VCD 저장 구문이 없다.
실행 시간을 지정하고 도구에 맞는 파형 저장 설정을 사용한다.
4. 생성한 파형에서 a,b,x,y,z를 확인하고 예상값과 비교한다.
5. 실행 방법·코드 설명·파형 해석을 실험 전 레포트에 쓴다.

VS Code 환경설정 매뉴얼 · [레포트 안내](#)

사전 단계는 이번 수업의 보충 안내이며 원본 Vivado 화면과 구분한다.

보충 · 보드 결과와 실험 후 레포트

1. Vivado 파형을 VS Code 사전 시뮬레이션과 비교한다.
2. 생성한 logic_gate.bit를 실제 FPGA에 기록한다.
3. SW_1·SW_2의 네 입력 조합과 LED1·LED2·LED3를 확인한다.
4. 연결 상태는 사진, 입력에 따른 출력 변화는 영상으로 남긴다.
5. 코드·파형·사진·영상을 GitHub에 올리고 레포트에 연결한다.

실제 결과와 예상값을 비교하고 오류·수정 사항을 설명한다.

원본 화면의 성공 표시와 본인의 실행 결과를 구분한다.

[레거시 01 처음으로](#) · [전체 목차 PDF](#)